

When Shared Experience Becomes an Implementation Attractor: Lock-In, Mitigation, and Task Boundaries in Parallel LLM Agents

Anonymous submission

Abstract

Parallel LLM agents are often made more capable by broadcasting prior outputs or manager-synthesized experience to later agents. We audit a risk that is weaker than catastrophic failure but easier to miss: shared experience may suppress independent solutions and stabilize a common implementation attractor. We instantiate a layered Mixture-of-Agents coding system with three private MiniSWE workspaces per layer, blinded experience transfer, and hidden executable evaluation. In a frozen 12-layer, three-arm Kimi-K2.6 experiment on a 37-test SlopCodeBench task, cumulative self-reported experience does not reproduce the preregistered sustained degradation curve. Instead, all workers share the same failed-test signature from layer 3 and the same exact implementation from layer 5, remaining at 33/37 through layer 12. Retrospective audits of two independent cumulative trajectories reproduce consecutive exact-snapshot lock-in from layers 3 and 4. A prospectively frozen fixed-three-reference control produces one primary implementation from layers 2–6, showing that cumulative history growth is unnecessary. A local-only diversity router increases post-layer-1 diversity AUC from 5 to 6 and delays lock-in from layer 2 to layer 3 without changing 34/37 quality, but does not prevent it. On a prospectively frozen same-provider `log_query` boundary, all arms retain three implementations through layer 3, while transfer improves mean all-test count from 108.4/134 to 127.2/134 and 130.1/134. Experience therefore creates a conditional quality-diversity trade-off rather than a universal multi-agent “death spiral.” A local completion-and-smoke gate additionally blocks two zero-command false completions, but rejects five strong solutions at its frozen threshold.

1 Introduction

Army ants can form a circular mill when locally sensible trail-following becomes self-reinforcing at the colony level (Schneirla 1944). The analogy is tempting for multi-agent LLM systems: each imported lesson can be plausible, each recipient can appear successful, and yet repeated sharing can remove the independence that made parallel agents useful. The analogy, however, is not evidence. This paper asks a narrower, executable question: *what actually happens*

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

when parallel coding agents repeatedly absorb shared experience?

Layered Mixture-of-Agents (MoA) systems let several agents solve the same task and then expose later agents to earlier outputs (Wang et al. 2024). Experience-driven agents similarly distill trajectories into reusable textual guidance (Zhao et al. 2024; Shinn et al. 2023), and multi-agent development systems share messages, roles, or cross-task experience (Qian et al. 2024b,a; Li et al. 2025). These mechanisms can improve accuracy and efficiency. They can also correlate recipients: once a privileged shared artifact is accepted, later “independent” workers may inherit the same assumptions, tests, and omissions.

Recent work has identified semantic anchoring and diversity collapse in multi-agent discussion (Guo et al. 2026; Zhang et al. 2026). Our focus is complementary. We study *experience-mediated lock-in* in an executable workflow where each agent edits a private repository, external tests measure exact behavior, and implementation identity can be audited by snapshot hash. This setting separates three outcomes that are often conflated: (i) performance collapse, (ii) implementation lock-in, and (iii) behavioral lock-in despite distinct code.

We initially sought a sustained degradation curve: as accepted experience accumulates, correct but increasingly complex guidance might trigger false completion and then propagate it. The strongest frozen replication rejects that story. The more reproducible finding is subtler. Cumulative experience rapidly homogenizes behavior and sometimes exact code, while catastrophic outcomes remain stochastic, model-dependent, and recoverable. A local admission rule blocks the observed zero-command false completions, but its threshold also discards strong solutions. Independent trajectory audits reproduce exact implementation lock-in, and a fixed-three-reference control shows that it can occur without a growing history. Recursive synthesis can lock agents onto an inferior fixed point on one task and improve them on another. A local-only diversity router delays rather than prevents lock-in on `cfgpipe`, while a same-provider `log_query` probe preserves maximal implementation diversity and improves quality under transfer.

Our contributions are:

- an audited layered MoA coding testbed with private workspaces, barrier synchronization, blinded transfer,

hidden external evaluation, and an exact experience-artifact specification;

- a preregistered negative result showing that depth and context growth do not reproducibly yield a sustained degradation curve across three model families;
- replicated trajectory and source-alignment evidence of an implementation attractor, plus a prospective fixed-reference control showing that cumulative history growth is not required in the tested setting;
- a prospective local-diversity router that yields only a one-layer delay, together with a same-provider second-task boundary where transfer improves quality without reducing implementation diversity; and
- a trajectory-level admission audit that quantifies the trade-off between rejecting false completion and retaining high-quality experience.

2 Related Work

Experiential agents and shared memory. ExpeL distills reusable insights from agent trajectories (Zhao et al. 2024); Reflexion stores verbal feedback for subsequent attempts (Shinn et al. 2023); Generative Agents and MemoryBank manage longer-term retrieval and salience (Park et al. 2023; Zhong et al. 2024). Experiential co-learning extends this idea to software-development teams (Qian et al. 2024a), while MAEL maintains cross-task experience in a collaboration graph (Li et al. 2025). These works motivate transfer as a capability mechanism. We audit what happens when accepted experience becomes a shared, recursively reused coordination state.

Parallel and role-based multi-agent systems. MoA uses layered parallel proposers and an aggregator (Wang et al. 2024). ChatDev and MetaGPT coordinate specialized software roles (Qian et al. 2024b; Hong et al. 2024); AgentVerse and MultiAgentBench study collaborative and competitive interaction (Chen et al. 2023; Zhu et al. 2025). Multi-agent debate aggregates reasoning rather than persistent experience (Du et al. 2024). Our workers instead solve the same coding task in private workspaces, so diversity is operational: agents may produce different repository states that pass and fail different tests.

Conformity and executable coding evaluation. Semantic anchoring can manufacture agreement and reduce diversity in LLM collectives (Guo et al. 2026; Zhang et al. 2026). We connect that concern to executable consequences by measuring exact snapshot hashes and failed-test sets. SlopCodeBench evaluates coding agents against iteratively revealed specifications and hidden executable tests (Orlanski et al. 2026). Its checkpointed projects provide a controlled middle ground between toy reasoning tasks and unconstrained repository issues such as SWE-bench (Jimenez et al. 2024).

3 Audited Parallel-Agent Testbed

3.1 Topology and Isolation

Figure 1 shows the fixed topology. At each layer, three MiniSWE workers receive the same public task and run concur-

Table 1: Transfer and admission policies. “Hidden” means external evaluator outcomes are withheld from recipients.

Policy	Shared state	Admission signal
No transfer	Empty	None
Self-report	All accepted summaries to date	Claimed success
Bounded recent	Three latest accepted summaries	Claimed success
Bounded diverse	Latest three locally distinct implementations	Claimed success; local source hash
Local admission	All accepted summaries to date	Claimed success and ≥ 3 successful commands
Recursive manager	One revised playbook	Self-report or external Core gate, depending on probe

rently in private repository workspaces. A barrier closes the layer before any output is shared. The next layer receives a policy-selected experience state; no worker sees another worker’s live workspace. An external evaluator then runs the frozen tests on each snapshot. External pass counts, failed-test names, and collection hashes are withheld from workers and from self-report or local-admission recipients.

The implementation uses AutoGen Core’s routed-agent runtime for the barrier and message topology, and the pinned native MiniSWE runner from SlopCodeBench for agent-environment interaction. We remove reference solutions, later checkpoint specifications, and hidden evaluator paths from worker-visible workspaces. Every interpreted result must match the frozen test count and collection hash, contain a valid model query, and pass a prohibited-access scan.

3.2 Transfer Policies

Table 1 defines the policies. **No transfer** suppresses all prior outputs. **Cumulative self-report** admits every output whose trajectory claims successful completion and appends its bounded experience summary to the shared history. **Bounded recent transfer** uses the same admission signal but retains only the three most recent accepted outputs. **Bounded diverse transfer** retains the latest three locally distinct implementation identities across bootstrap and completed layers. **Local admission** additionally requires at least three successful environment commands; this is an agent-local completion-and-smoke proxy, not external verification. **Recursive synthesis** sends accepted outputs and the previous shared playbook to a manager, which emits one revised playbook for the next layer.

3.3 Experience Artifacts and Local Signals

Each worker emits a bounded textual artifact with three fields: a worker/layer header, the trailing 6,000 characters from its

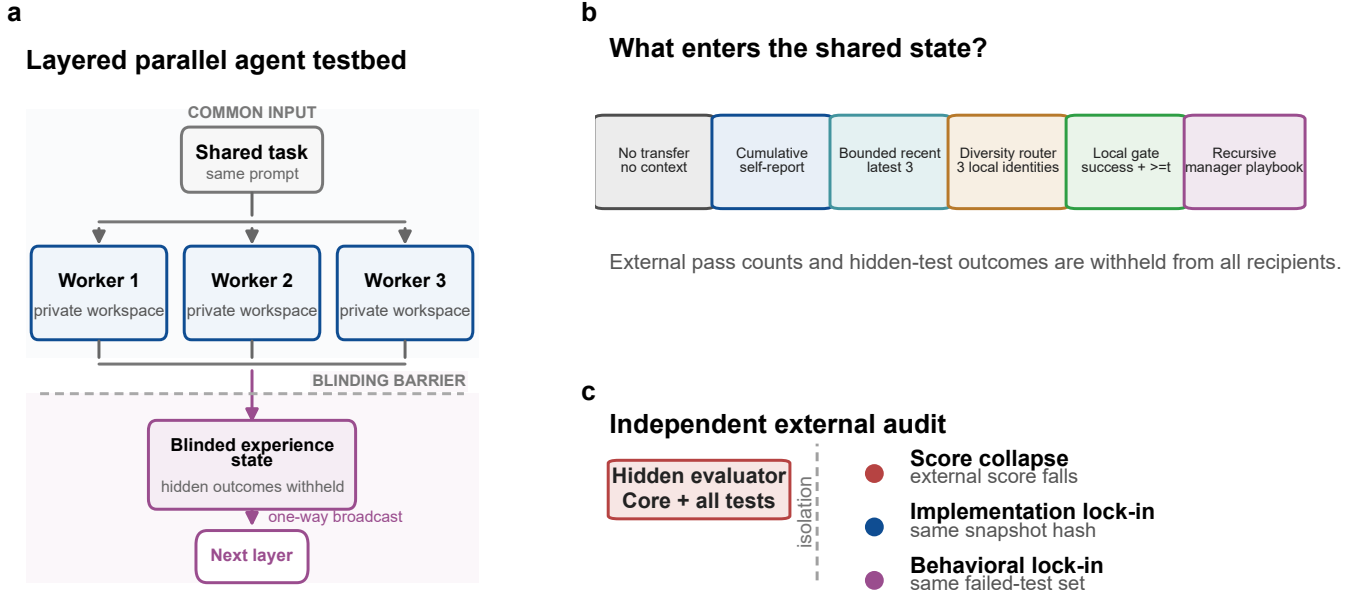


Figure 1: **Audited layered parallel-agent testbed.** **a**, Three MiniSWE workers solve the same task concurrently in private workspaces; a barrier closes the layer before a blinded experience state is broadcast. **b**, Policies differ only in what enters that state: nothing, self-reported success, a fixed recent window, a local implementation-diversity router, self-report plus a local command threshold, or one recursively synthesized manager playbook. **c**, A hidden evaluator separately measures external score, implementation identity, and failed-test behavior. External outcomes are never delivered to self-report or local-admission recipients.

final four assistant steps, and a UTF-8 source-snapshot excerpt, followed by whole-artifact truncation at 12,000 characters. The recipient prompt labels transferred artifacts as fallible evidence and independently truncates each source at the same limit. In the frozen main cumulative arm, the median artifact is 10,479 characters; median received experience grows from 25,934 characters at layer 1 to 367,725 at layer 12.

Self-report admission scans the concatenation of all assistant messages using case-insensitive word-boundary matching over pass, passes, passed, works, working, success, successful, correct, verify, and verified. The local gate adds at least three successful environment commands. Neither rule reads the external evaluator. The diversity router hashes local implementation files while excluding test files, caches, and evaluator output; it introduces no model call or hidden score. The exact regular expression is provided in the supplement.

3.4 Frozen Main Experiment

The main experiment uses Kimi-K2.6 through one MaaS provider on `cfgpipe` checkpoint 1: 37 executable tests, including a four-test Core subset. Each arm has three workers, 12 layers, a 12-action limit, temperature 0.6, and a 12,000-character cap per source. The two transfer arms start from the same three frozen, diverse sources, each independently re-evaluated at 4/4 Core and 34/37 overall; their external scores are withheld in recipient prompts. No transfer receives no

source, by definition.

The protocol freezes three primary questions before new-arm outcomes: (i) whether cumulative self-report produces at least two late 0/37 events, (ii) whether no transfer has fewer such events, and (iii) whether local admission rejects every zero-command 0/37 event and prevents its direct downstream appearance. All 108 worker evaluations pass target consistency, model-query integrity, prompt blinding, and prohibited-access gates.

3.5 Prospective Controls and Independent Audits

We retrospectively apply the same snapshot and failed-test audit to two independent cumulative self-report trajectories beyond the frozen main run: a 10-layer Kimi Coding run and a 12-layer Kimi MaaS run. The lock-in metric was not pre-registered for these trajectories, so they are replication audits rather than independent inferential samples. An earlier Coding trajectory exposed bootstrap Core scores and is retained only as a protocol-drift diagnostic.

We additionally freeze a fresh six-layer MaaS control before observing its outcomes. It matches the main self-report setting but broadcasts only the three most recent accepted summaries at every layer. The prespecified criterion is two consecutive layers in which all three valid snapshots are byte-identical. This tests whether cumulative reference-count growth is necessary; it does not equalize all input tokens with no transfer.

After auditing that control, we freeze a local-only

diversity-preserving router before its outcomes. The `cfgpipe` protocol reuses the already frozen no-transfer and bounded-recent controls and runs only the new six-layer arm; this control reuse and the known control outcomes are disclosed in the protocol. Its primary endpoint is primary-implementation diversity AUC over layers 2–6. We then freeze a compact same-provider `log_query` three-arm boundary with three layers, identical bootstrap sources for both transfer arms, and no outcome-dependent stopping.

3.6 Mechanism Probes and Boundaries

We analyze two recursive-manager stress probes on `cfgpipe`. A Kimi probe admits only 4/4 Core outputs to the manager and includes matched no transfer. A DeepSeek-V4-Flash probe admits self-reported success and deliberately bootstraps from a complete, score-blinded cohort whose audit-only outcomes are 0/37, 33/37, and 34/37. The latter tests compression of mixed-success experience; it cannot establish amplification of globally correct experience.

Boundary runs apply recursive synthesis to `log_query`, `xjq`, and sequential `cfgpipe` or `execution_server` checkpoints. Runs with action-boundary-censored controls remain descriptive and are not used for a confirmatory effect claim. A separate `log_query` boundary compares strictly Core-gated cumulative transfer with no transfer and audits terminal snapshot diversity.

3.7 Metrics and Decision Discipline

For worker i at layer ℓ , external score is $s_{i\ell} = p_{i\ell}/m$, where p is the number of passed tests and m is the frozen collection size. We report layer means and worker ranges. A catastrophic event is exactly 0/ m .

Implementation diversity is reported at two levels. A full-workspace hash covers all submitted files. A primary-implementation hash covers the task entry point (`cfgpipe.py` or `logql.py`) and prevents inert scratch or test files from being mistaken for a new implementation. The prospective routing and boundary endpoints use primary hashes; historical replications retain their pre-existing full-workspace audit. Behavioral diversity is computed from each worker’s failed-test set $F_{i\ell}$: we report the number of unique signatures, dominant-signature share, and mean pairwise Jaccard similarity

$$J_{\ell} = \frac{1}{3} \sum_{i < j} \frac{|F_{i\ell} \cap F_{j\ell}|}{|F_{i\ell} \cup F_{j\ell}|}.$$

Coordination load is measured by received references, experience characters, and input tokens relative to no transfer.

For mechanism auditing, we also compare each recipient implementation with all delivered prior sources using exact primary-file match, normalized source-line Jaccard, function-name overlap, AST-node-count signatures, string literals, and verbatim line availability in the delivered snapshot excerpt. These are alignment measures, not causal estimators of copying.

Each run is one coupled trajectory; layers are not independent seeds. We therefore use descriptive exact counts rather than pseudo-replicated significance tests. The two historical

lock-in replications were audited retrospectively, and cross-model or cross-provider runs are reported separately rather than pooled as exchangeable trials.

4 Results

4.1 RQ1: Sustained Degradation Is Not Reproduced

Figure 2a shows the frozen 12-layer result. Cumulative self-report moves from 34/37 at layer 1 to a three-worker mean of 33/37 at layer 3, then remains exactly at 33/37 through layer 12. It produces no 0/37 events. Thus the preregistered recurrent catastrophic-tail replication is falsified.

Neither control supports a unique transfer-induced catastrophe. No transfer has valid action-budget tails of 12/37 and 1/37 at layers 5 and 6, followed by recovery. Local admission has one 0/37 worker at layer 10 and another at layer 11, so each layer mean falls to 0.613, then all workers return to 34/37 at layer 12. The two local-arm events are independent false-completion trajectories rather than propagated snapshots: each uses one model step, executes zero commands, produces no snapshot, and is rejected.

Across raw cumulative self-report runs, DeepSeek-V4-Flash has 2/30 zero-score events, GLM-5 has 0/30, and Kimi-K2.6 has 1/24 in an initial coding-endpoint run, 0/30 in a preregistered replication, and 0/36 in the fresh MaaS replication (Figure 3b). Every observed valid zero event is isolated or terminal; none establishes a persistent downward curve. More layers increase the opportunity to sample rare agent failures, but depth alone is not a sufficient explanation for sustained degradation.

4.2 RQ2: Shared Experience Removes Independent Solutions

The absence of a death spiral does not imply that transfer is inert. In cumulative self-report, all three workers share one failed-test signature from layer 3 through layer 12. At layers 3–4 they occupy two snapshot hashes; from layer 5 onward all 24 remaining outputs are byte-identical directory snapshots. Every layer-3–12 worker passes Core and fails the same four broader tests.

At layer 12, no transfer retains three distinct snapshots, three failed-test signatures, dominant snapshot share 1/3, and pairwise failed-test Jaccard 0.278. Both cumulative self-report and local admission end with one snapshot, one signature, dominant share 1, and Jaccard 1. Local admission therefore blocks some trajectories but does not preserve terminal implementation diversity among accepted trajectories.

The same implementation phenomenon recurs outside the main trajectory (Figure 2b). The independent Coding run has three distinct layer-2 snapshots and one shared snapshot from layers 3–10. The independent MaaS run reaches one snapshot from layers 4–9, experiences an intermittent no-snapshot 0/37 event at layer 10, and reconverges at layer 11. Neither matched no-transfer control has two consecutive exact-convergence layers.

The prospective bounded arm separates lock-in from cumulative history growth (Figure 2c). Layer 1 contains three distinct 34/37 primary implementations. From layer 2

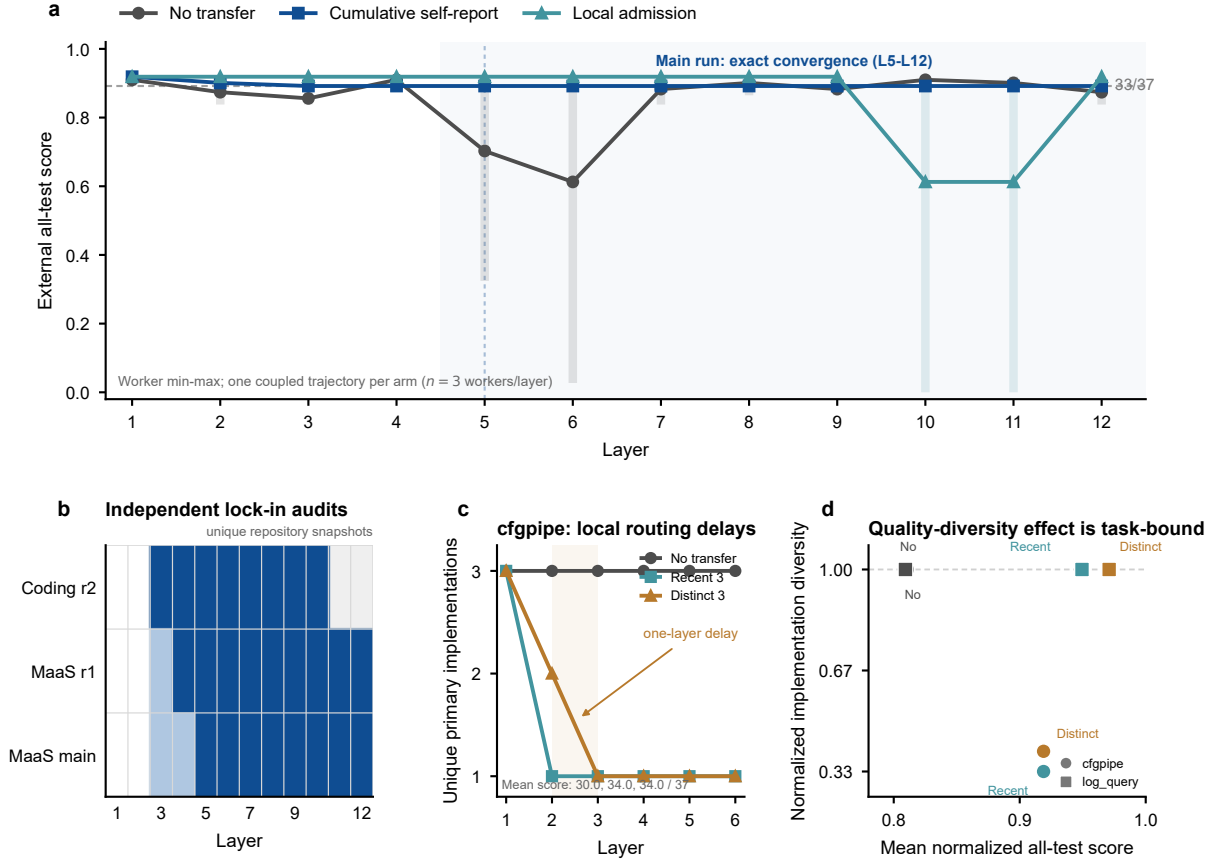


Figure 2: **The implementation attractor is reproducible, weakly mitigated, and task-bound.** **a**, Frozen main-run external score; ranges show three-worker minima and maxima. **b**, Unique full-workspace snapshots in two retrospective cumulative replications and the frozen main run; dark cells denote exact three-worker convergence. **c**, Prospective cfpipe primary-implementation diversity. Fixed-recent routing locks at layer 2; the local-only diversity router retains two implementations at layer 2 but locks from layer 3, with no quality change. **d**, Mean normalized score versus normalized post-layer-1 primary-implementation diversity. Transfer improves score on both tasks, but reduces diversity only on cfpipe. Circles denote cfpipe and squares denote the prospectively frozen same-provider `log_query` boundary. All panels are descriptive coupled trajectories, not independent seeds.

through layer 6, all workers produce the same `cfpipe.py` with exactly three references per worker. Full-workspace hashes differ at layers 4 and 6 only because of an auxiliary `ws.txt` or test file; they do not represent primary-implementation re-diversification. At the first lock-in layers, total input is 1.26–1.60 times the matched no-transfer load, versus 14.08 times at cumulative layer 12. Accumulating dozens of references is therefore unnecessary, although transferred context remains a plausible contributor.

The delivered artifacts contain unusually direct implementation evidence. After layer 1, 87.9% of cumulative recipients exactly match at least one prior primary implementation; their mean maximum source-line Jaccard is 0.965, and 97.6% of recipient source lines are available verbatim in at least one delivered snapshot excerpt. In the bounded arm all three values are 1.0. These measurements support an implementation-attractor mechanism, while not distinguishing literal copying from deterministic regeneration condi-

tioned on the same code-rich prompt.

4.3 RQ3: Local Diversity Routing Delays but Does Not Prevent Lock-In

The prospective local-only router preserves the three-reference cap but selects the most recent three distinct implementation identities across all prior layers. On cfpipe, bounded-recent diversity AUC over layers 2–6 is 5, the minimum possible value, and exact primary lock-in begins at layer 2. Diversity-aware routing raises AUC to 6 by retaining two implementations at layer 2, then locks from layer 3 through layer 6. Both transfer arms score 34/37 for all 18 outputs, so this is a one-layer delay without a quality trade-off, not a solution.

The prospectively frozen same-provider `log_query` probe is a boundary rather than a replication (Figure 2d). No transfer, bounded recent, and bounded diverse each retain three primary implementations at every measured

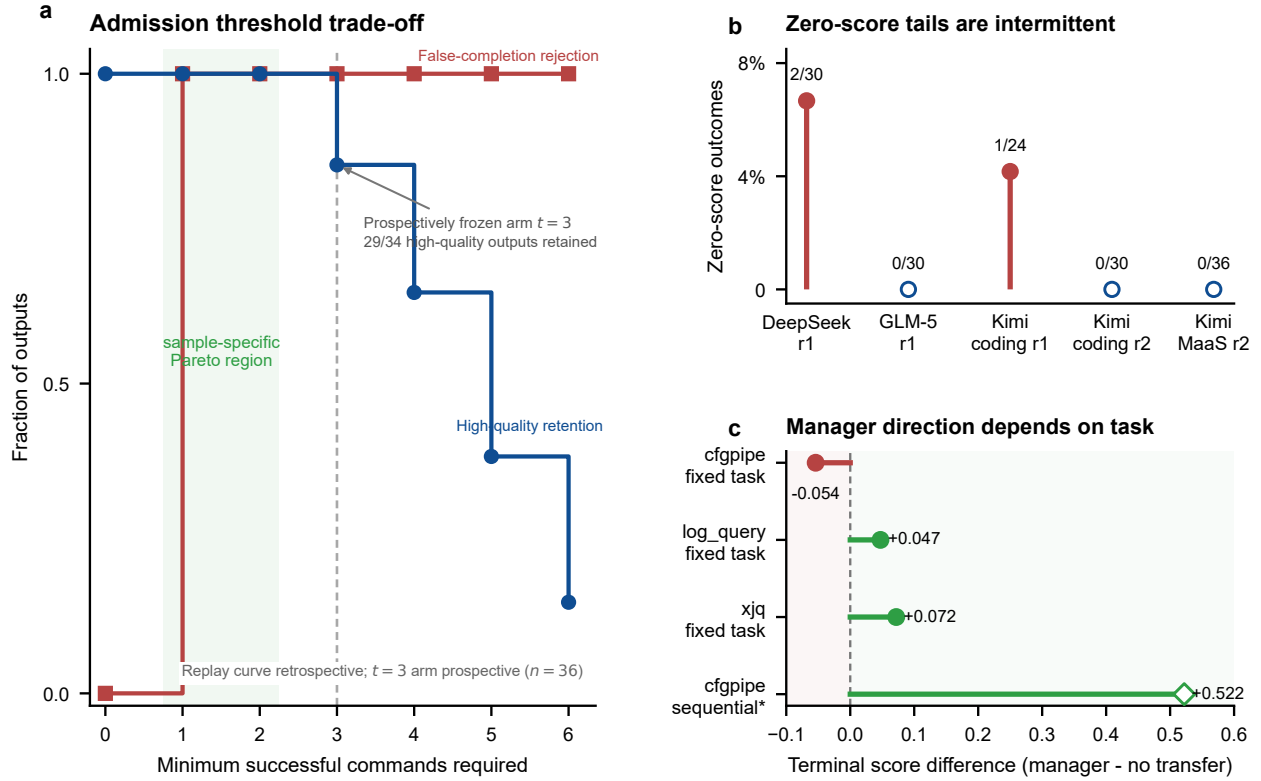


Figure 3: **Admission and outcome direction are conditional.** **a**, Retrospective threshold replay over all 36 local-arm outputs: catastrophic rejection is the fraction of two 0/37 outputs rejected, and high-quality retention is the fraction of 34 outputs scoring at least 33/37 that are admitted. The replay curve is retrospective, while the marked three-command arm was prospectively frozen before its outcomes; the shaded thresholds remain sample-specific rather than validated optima. **b**, Valid 0-score event counts under cumulative self-report across five audited model runs (150 worker outcomes total); rates are shown separately because layers and providers are not exchangeable. **c**, Descriptive terminal treatment-minus-no-transfer score differences for recursive-manager settings. The sequential `cfpipe` control is noisy and marked with an asterisk; effects are not pooled.

layer. Mean all-test counts are 108.4/134, 127.2/134, and 130.1/134, respectively, while the two transfer arms both average 9.67/10 Core. Because bounded recent already reaches the diversity ceiling, no incremental mitigation effect is identifiable. The `cfpipe` attractor is therefore conditional on task and trajectory rather than an automatic consequence of shared experience.

4.4 RQ4: Recursive Synthesis Can Privilege a Fixed Point

The recursive-manager probes show how a compact privileged artifact can stabilize shared behavior. In the Kimi `cfpipe` probe, the manager’s layer-3 and layer-4 playbooks are byte-identical. At layer 5, all recursive workers produce the same snapshot and score 32/37, whereas no-transfer workers produce three different snapshots and score 34/37. The manager repeatedly preserves an incorrect source-precedence rule and two additional edge-case errors. This supports a fixed-point lock-in mechanism, but the preregistered primary Core degradation criterion is falsified because both arms retain 4/4 Core.

The DeepSeek stress probe reaches a near-textual manager

fixed point. Its three terminal implementations remain distinct, yet all score 32/37 and fail the same five tests. Because the score-blinded bootstrap includes an externally failed source, this is evidence that synthesis compresses mixed-success trajectories into common behavior, not that locally or globally correct experience becomes harmful by itself.

The direction does not generalize across tasks (Figure 3c). Recursive synthesis improves terminal normalized all-test score by 0.047 on `log_query` and 0.072 on `xjq`. A sequential `cfpipe` pilot also favors treatment by 0.522 against a noisy action-budget control. An `execution_server` pilot is descriptively positive but confirmatorily invalid because no transfer reaches the frozen action boundary without explicit completion. Shared fixed points can therefore encode useful stabilization or shared error; task and admission semantics determine the direction.

4.5 RQ5: Admission Blocks False Completion at a Cost

The local gate’s three-command threshold was frozen before the 36-output arm was run. It rejects both observed 0/37 false completions and their artifact identifiers never occur

in later recipient prompts. This is a prospective mechanical protection against direct propagation in the audited arm.

The same gate rejects seven of 36 outputs: the two 0/37 events and five externally strong 34/37 outputs that use only two successful commands. We replay thresholds 0–6 on the frozen trajectories (Figure 3a). Thresholds 1 or 2 retain all 34 high-quality outputs while rejecting both zero-command events in this sample; the frozen threshold 3 retains 29/34 high-quality outputs. This is a retrospective sensitivity result, not evidence that two commands is universally optimal. It shows that admission quality depends on threshold calibration and that “more verification” is not cost-free.

5 Discussion

The main risk is loss of optionality. Parallel agents are valuable partly because independent trajectories can explore different implementations and failure modes. In the main run, transfer removes that optionality long before it creates catastrophic score loss. A system that reports only mean success may therefore look healthy while all workers share the same blind spot. Implementation and failed-test diversity are useful companion metrics for multi-agent coding systems.

Stability and diversity are not monotonic goods. The same-provider boundary shows why homogenization cannot be labeled inevitable or harmful in general. On `log_query`, shared experience raises score without reducing implementation diversity. The engineering objective is not maximal diversity; it is *validated pluralism*: preserve meaningfully different candidates until evidence is strong enough to collapse them.

Duplicate suppression is not sufficient mitigation. The local diversity router changes one mechanism variable without hidden evaluation or extra model calls, yet buys only one layer on `cfgpipe`. Once all recipients see the same code-rich bundle, distinct sources can still induce the same preferred implementation. Stronger mitigations may require recipient heterogeneity, counterfactual task partitions, disagreement-seeking prompts, or explicit portfolio objectives rather than routing alone.

Admission is a governance problem. Self-report admits obvious false completion. A command-count gate catches the observed zero-command cases but confuses concise work with absent work. Better admission should combine provenance, task-relevant local tests, change evidence, uncertainty, and diversity contribution. It should also retain rejected artifacts for audit rather than silently erasing them.

More layers are not always a better mechanism test. Once the shared playbook or snapshots reach a fixed point, additional identical layers mostly resample stochastic agent execution and inflate context. A stronger causal test changes one mechanism variable—admission, compression, routing, or recipient heterogeneity—under a matched control, rather than extending an already stationary curve.

6 Limitations

The strongest confirmatory result remains one frozen Kimi provider run on one `cfgpipe` checkpoint with three work-

ers per layer. Two independent cumulative trajectories reproduce exact lock-in retrospectively, but they were not pre-registered for that metric and do not provide an inferential sampling distribution. The bounded-context control is one prospective trajectory and does not equalize every input token. The diversity-router arm reuses already observed frozen controls, has one trajectory, and tests only exact local identity deduplication. The same-provider `log_query` boundary is prospective but only three layers deep, so later lock-in remains possible. Neither prospective probe provides independent seeds, larger worker cohorts, temperature sweeps, or heterogeneous models. Cross-model audits use different provider endpoints and are not statistical replications of one immutable model binary. SlopCodeBench tasks are executable and realistic enough to expose coding behavior, but they do not cover browser work, long-lived organizations, or asynchronous agents. The recursive probes differ in admission semantics; the DeepSeek probe has a mixed-quality bootstrap, and some sequential controls are action-budget noisy or censored. The gate arm is prospective, but the threshold sweep is retrospective. Full-workspace equality can overstate implementation change when harmless auxiliary files differ; we therefore use primary entrypoint hashes for new prospective endpoints, but historical replications retain full snapshots. Failed-test equality can still understate latent differences not covered by the evaluator. We therefore claim a bounded, audited failure mode and trade-off, not a universal law of multi-agent learning.

7 Conclusion

Repeated shared experience did not produce the sustained ant-mill-style degradation curve we sought. It produced a more defensible result: parallel coding agents can rapidly lose implementation and behavioral independence, repeatedly enter a shared implementation attractor, and still maintain apparently strong aggregate scores. Exact lock-in recurs across three cumulative trajectories and appears under a fixed-three-reference control, so cumulative history growth is not necessary in the tested setting. Local identity-aware routing delays lock-in by one layer but does not prevent it. The prospectively frozen same-provider `log_query` boundary retains maximal implementation diversity while transfer improves quality, showing that the attractor is task-conditional rather than universal. Catastrophic tails are intermittent and model-dependent; recursive synthesis can help or hurt; and local admission blocks false completion only by choosing a threshold with false-rejection cost. Multi-agent systems should therefore evaluate not only whether shared experience improves the mean, but also what diversity it removes, what evidence admits it, and whether mitigation remains effective beyond the next layer.

References

Chen, W.; Su, Y.; Zuo, J.; Yang, C.; Yuan, C.; Chan, C.-M.; Yu, H.; Lu, Y.; Hung, Y.-H.; Qian, C.; Qin, Y.; Cong, X.; Xie, R.; Liu, Z.; Sun, M.; and Zhou, J. 2023. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. arXiv:2308.10848.

- Du, Y.; Li, S.; Torralba, A.; Tenenbaum, J. B.; and Mordatch, I. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *International Conference on Machine Learning (ICML)*.
- Guo, Y.; Chen, S.; Wu, C.; Ma, W.; and He, B. 2026. Diversity Collapse in Multi-Agent LLM Systems: How Semantic Anchoring Amplifies Bias and Manufactured Consensus. arXiv:2604.21879.
- Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Zhang, C.; Wang, J.; Wang, Z.; Yau, S. K. S.; Lin, Z.; Zhou, L.; Ran, C.; Xiao, L.; Wu, C.; and Schmidhuber, J. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*.
- Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. R. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations*.
- Li, Y.; Zhang, Q.; Jiang, S.; Shao, B.; Yu, Z.; Deng, X.; Weng, Y.; Lin, Z.; Tang, Y.; Huang, C.; and Zhang, W. 2025. Cross-Task Experiential Learning on LLM-Based Multi-Agent Collaboration. arXiv:2505.23187.
- Orlanski, G.; Baranwal, A.; Xiao, Y.; Yao, G.; and Baghaddadi, R. 2026. Learning to Write Useful Code from Iterative Specifications. arXiv:2603.24755.
- Park, J. S.; O'Brien, J. C.; Cai, C. J.; Morris, M. R.; Liang, P.; and Bernstein, M. S. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.
- Qian, C.; Dang, Y.; Li, J.; Liu, W.; Chen, W.; Yang, C.; Liu, Z.; and Sun, M. 2024a. Experiential Co-Learning of Software-Developing Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Qian, C.; Liu, W.; Liu, H.; Chen, N.; Dang, Y.; Li, J.; Yang, C.; Chen, W.; Su, Y.; Cong, X.; Xu, J.; Li, D.; Liu, Z.; and Sun, M. 2024b. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Schneirla, T. C. 1944. A Unique Case of Circular Milling in Ants, Considered in Relation to Trail Following and the General Problem of Orientation. *American Museum Novitates*, 1253: 1–9.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Wang, J.; Wang, J.; Athiwaratkun, B.; Zhang, C.; and Zou, J. 2024. Mixture-of-Agents Enhances Large Language Model Capabilities. arXiv:2406.04692.
- Zhang, Z.; Zhang, W.; Luo, H.; and Awad, E. 2026. Multi-LLM Systems Exhibit Robust Semantic Collapse: When Diversity Guarantees Fail. arXiv:2606.01448.
- Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.-J.; and Huang, G. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 19632–19642.
- Zhong, W.; Guo, L.; Gao, Q.; Ye, H.; and Wang, Y. 2024. MemoryBank: Enhancing Large Language Models with Long-Term Memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Zhu, K.; Du, H.; Hong, Z.; Yang, X.; Guo, S.; Wang, Z.; Wang, Z.; Qian, C.; Tang, X.; Ji, H.; and You, J. 2025. Multi-AgentBench: Evaluating the Collaboration and Competition of LLM Agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.